Towards Re-Engineering Digital Twins

Preliminary Experiments on Three Use Cases

Gwendal Beaumont*
gwendal.beaumont@imt-atlantique.fr
IMT Atlantique
Lab-STICC, UMR 6285
Brest, France

Antoine Beugnard
antoine.beugnard@imt-atlantique.fr
IMT Atlantique
Lab-STICC, UMR 6285
Brest, France

Salvador Martínez
salvador.martinez@imt-atlantique.fr
IMT Atlantique
Lab-STICC, UMR 6285
Brest, France

Christelle Urtado
Christelle.Urtado@mines-ales.fr
EuroMov Digital Health in Motion
Univ. Montpellier & IMT Mines Ales
Ales, France

Sylvain Vauttier
Sylvain.Vauttier@mines-ales.fr
EuroMov Digital Health in Motion
Univ. Montpellier & IMT Mines Ales
Ales, France

Abstract

Digital Twins (DTs) are everywhere, but they often have been developed before consensual definitions, reference models or adapted tools were proposed. Thus, these *ad hoc* DTs cannot be easily manipulated as actual *entities*, hampering both their sustainability and their life cycle management (*e.g.*, their redeployment). In order to deal with this issue, this paper chooses to learn from practice by re-engineering existing DTs. Starting from a generic *a priori* metamodel that tries to clearly separate concerns (so that DTs can be seen as first class software entities), this paper implements experiments in three steps: (i) reverse engineering an existing Digital Twin, seen as a prototypical instance, (ii) generalization of the DT architecture descriptor into a DT model, (iii) alignment between this reverse engineered DT model and the *a priori* DT metamodel. First experiments, run on three use cases, show that our proposal can relevantly analyze and document DTs to support their re-engineering as first class entities.

CCS Concepts

• **Software and its engineering** → **Designing software**; • **Computer systems organization** → **Embedded and cyber-physical systems**; • **Information systems** → *Information systems applications.*

Keywords

Digital Twin, Metamodel, Model-Driven Engineering, Digital Twin Architecture, Reverse Engineering Digital Twins, Re-Engineering Digital Twins.

*Also with EuroMov Digital Health in MotionUniv. Montpellier & IMT Mines Ales,.

MODELS Companion '24, Linz, Austria
© 2024 Copyright held by the owner/author(s).
This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion '24), September 22–27, 2024, Linz, Austria*, https://doi.org/10.1145/3652620.3688259.

ACM Reference Format:
Gwendal Beaumont, Antoine Beugnard, Salvador Martínez, Christelle Urtado, and Sylvain Vauttier. 2024. Towards Re-Engineering Digital Twins: Preliminary Experiments on Three Use Cases. In *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion '24), September 22–27, 2024, Linz, Austria.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3652620.3688259

1 Introduction

Digital Twins (DTs) are everywhere. Indeed, the DT *paradigm* has been adopted in a broad range of domains and with different purposes, with the general goal of taming the complexity of Cyber-Physical Systems (CPSs) [5, 20]. However, from a software engineering perspective, the situation remains far from ideal [3, 7]. DTs are often developed in an *ad hoc* manner, as an amalgam of sensors information, communication components, analysis tools, and configuration scripts, tightly coupled to underlying technologies and execution platforms. As a consequence, once developed and deployed, life cycle management and other operations (*e.g.*, reuse, redeployment, composition, etc.) on DTs are very difficult, requiring low-level platform-specific knowledge. We recently have faced this problem when trying to deal with three existing DTs [1, 2, 22] for their redeployment in a new in-house DT execution platform.
To alleviate this issue, many different frameworks have been proposed for the description and deployment of DTs [12, 19] but none of them has become a *de facto* standard yet. In the same way, while forward engineering approaches that derive (parts of) DTs from high-level specifications exist [4, 8, 17], they rarely focus on the life cycle of the DT. Conversely, this paper proposes an approach that integrates life cycle management as a core concern and focuses on re-engineering DTs.
A preliminary step of our approach consists in a conceptualization of DTs aiming to provide the required abstractions to represent DTs as first class software entities. Such software entities should encompass all the required information for its life cycle management, including a description of the knowledge (*i.e.*, data, models and metadata) both used by the DT and produced by the DT, the connections the DT is capable of, its configurations, and its responsibilities. Thereupon, we provide a semi-automatic reverse

engineering process thanks to which we bridge the gap between this conceptualization and already deployed DTs. At this point, the re-engineering of DTs is made easier, guided by the metamodel we propose.

Experiments have been conducted on three uses cases. Concretely, we re-engineer for their redeployment three different existing DTs: the DT of a robotic arm (sub-section 2.1), the DT of a connected apartment (sub-section 2.2), and the DT of a Mass-Spring Damper (sub-section 2.3).

The remainder of the paper is organized as follows. Section 2 presents the three motivating use cases. Section 3 provides rationale regarding the conceptualization of DTs, and a corresponding metamodel. Section 4 describes a process to create instances of the aforementioned metamodel by analyzing existing deployed DTs. Section 5 provides a brief analysis of our experiments while Section 6 discusses related work. Section 7 concludes and outlines future work.

2 Use cases

This section introduces the three use cases (UCs) corresponding to three different DTs. These UCs will be used throughout the paper to illustrate the challenges of re-engineering DTs and to experiment our approach.

2.1 UC1: Robotic Arm

The first UC is a Jaco® robotic arm from Kinova®. This arm has been designed to help disabled person in their everyday life.

The DT of this robotic arm is based upon the Robot Operation System (ROS), Unity, Gazebo and Docker. Here, ROS acts as a 'broker' between the robotic arm and the graphical model which can either be in Unity or Gazebo. When an action needs to be sent to the robotic arm, data flows as follows:

(1) First, a trajectory of the robotic arm is drawn in Unity or Gazebo and sent to ROS.
(2) The trajectory is analyzed by ROS and rectified to a feasible path if necessary.
(3) The (rectified) trajectory is sent by ROS to the robotic arm.
(4) The robotic arm moves.

The other way round, when the robotic arm moves, its (Unity) graphical representation updates to reflect the changes. The data from the sensors is sent to ROS which sends the updated position of the arm to the GUI through an API. Moreover, there are two ways of deploying this DT: "as is" on the machine, or using a Docker[1] container.

2.2 UC2: Human at home projecT (HUT)

The second UC is the DT of a connected apartment from the HUman at home projecT[2]. The goal of this innovative project is to explore and anticipate the effects of new technologies and connected devices on daily behaviors and well-being at home.

This DT is based upon Internet of Things (IoT) and web technologies such as: openHAB, Z-Wave, React and Docker. In our case, we do not have access to the apartment itself but to data gathered from the experiment and a few physical objects to replicate the actual apartment. These physical objects include a smart RGB light, a smart plug, a smart contact door sensor, and a 6-in-1 sensor.

This DT works as follows: an openHAB Docker container installed on a Raspberry Pi 4 acts as a middleware between the DT and the physical objects. Physical objects connect to openHAB using the Z-Wave protocol. A web interface using React connects to an API exposed by openHAB. As an example: when the user switches a light on the web interface, the switch order is sent to openHAB which, in turn, converts the order to an action on the virtual Thing; then this order is sent by openHAB to the real light. For updates from physical objects, the process is reversed.

2.3 UC3: Mass-Spring-Damper Model

The third UC of our study is the DT of the Mass-Spring Damper (MSD) model, a well-known textbook case[3]. This DT is only a simulation, therefore no reference system is connected to it. This DT works around two Functional Mock-up Unit (FMUs) files providing the models for the simulation, a `jar` file which can run the simulation, and JSON files containing the initial parameters for the tool. Furthermore, a folder called `strategies` contains different deployment strategies of the DT for Docker, Ansible and shell scripts.

3 Digital twin conceptualization

DTs have hundreds of definitions [7]. They also have dozens of usages [7]. They are used before the reference system exists (for its design), during its existence (for its control or monitoring) or after it has disappeared or been dismantled. They are also used for simulation, analysis, control, monitoring, prediction, learning etc. DTs can be applied to a lot of scientific domains: architecture, urban planning, ecology, industry, medicine, archaeology, physics etc.

The applicability is so large, that, to a certain extend, we could say that software engineering is about DTs. "Digitalizing" a system is programming a DT. A text editor is a DT of the printed sheet of paper. An information system is a DT of an organization.

Then, what does the DT paradigm bring to software engineering? Little, if we continue business as usual. But, if DTs were designed with specific rules, maybe software engineering would change. We propose two rules:

Rule 1: A DT does not contain any data exploitation tool. There are too many tools, too many usages to embed them into a DT. As a major software engineering principle, the single responsibility principle of a component (here, a DT) should be applied. We argue that the responsibility of a DT is to collect data, models, information knowledge, wisdom about something (a reference system). Tools are outside the DT. Tools get information from the DT and produce new information. The DT's responsibility is to manage this information, to contextualize it so that its source, its quality, its scope, and so on, are known. *A DT should be usage agnostic.* Conversely, *a tool should be business agnostic.* A tool is dedicated to some defined computation or analysis, whatever the business is. Such a separation of responsibilities would provide very interesting reuse capabilities where tools are *DT-aware* (Figure 1). Tools are configured with information found in the DT they work for.

[1] https://www.docker.com/
[2] https://www.hut-occitanie.eu/en/home/
[3] https://en.wikipedia.org/wiki/Mass-spring-damper_model

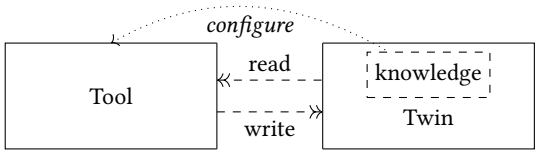

Figure 1: Digital Twin tool awareness

Rule 2: A Digital Twin is a knowledge management space. Focused on knowledge management [15], a DT has to organize, manage and *contextualize* data and information. This could be provided by dedicated DT services. DTs have to help generate, check, improve, and clean (meta)data. We share the vision of D. Stahl of a world where no data specialist ever need to access raw data [21]. In short, the responsibility of the DT is to *organize* data management of a reference system. Then, as in Eramo *et al.* [11], a DT does not only contain models of its reference systems, it also composed of models of its own organization, services, and life cycle.

With these principles in mind, we propose to reverse-engineer three DT implementations to try and separate responsibilities, to identify tools and the re-engineering required to make them DT-aware. As a guide, we introduce a DT conceptualization (see Figure 2).
This very simple metamodel, inspired from other works found in the literature [1, 2, 10], introduces the main concepts that need to be identified and organized. It depicts a `Digital Twin` as a kind of CPS, or even a socio-technical system (*i.e.*, including humans) `CPS/STS`. A DT has a CPS/STS as its reference system (`reference system` relationship) and can be used with many *external* `Tools`. `Tools` use (read, create, update) `Knowledge` through `Services`. The `Connection` produces `Knowledge` and relays commands from `Services`. The DT `Configurations` are part of the `Knowledge` as the `Tool Configurations` are.

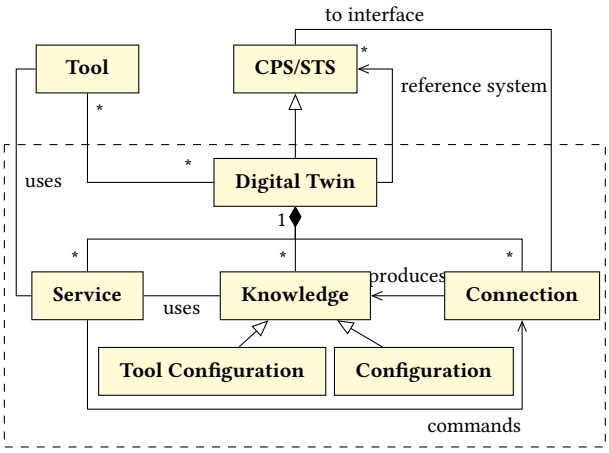

Figure 2: A very simple digital twin metamodel

We expect the DT of a system to be unique. This may seem like an ideal to strive towards, but we imagine converging towards an effective share of knowledge among DTs in the long term.

As presented in **Rule 1**, we choose to externalize tools, and then usages of DTs. We design DTs as agnostic towards their uses. A DT is a knowledge management system (**Rule 2**). Parts under the DT's responsibility are inside the dashed line that indicates the boundary of the DT in Figure 2.
A DT is composed of three parts:

(1) The `Connection` realizes the twinning, *i.e.*, coupling the reference system to its DT. The Connection is abstracted by some `Configuration` knowledge (*e.g.*, host and port addresses). The `Connection` is on the DT border and makes the link with the `CPS/STS`.
(2) `Services` describes the interface through which Tools (and users) can interact with the DT. It is a set of programs which ensure the consistency of the DT knowledge[4]. In particular, they trigger commands through the `Connection`.
(3) `Knowledge` includes data collected through the `Connection`, and processed by external `Tools`. It also contains models of any kind about the `CPS/STS`: structural, 3D, functional, behavioral, etc. `Configuration` describes versions, variants, instances of the reference system: it is the DT's specific knowledge. `Tool Configuration` contains all DT's specific information used by external tools to allow `Knowledge` exploitation: *e.g.*, a simulation configuration, or a data analysis algorithm. This can be used as metadata for data produced by external tools. Part of the tool configuration is obviously in `Knowledge`; the part that describes the `CPS/STS` only.

In the next section, we use this very abstract organization of a DT on use-cases we have reverse engineered.

4 Reverse Engineering

As mentioned in Section 3, we believe there is a need for the reverse engineering of DTs. We devote this section to the description of our proposed reverse engineering process. Then, we show the results of applying this process to the three use-cases described in Section 2 for their alignment to the metamodel proposed in Section 3.

4.1 Reverse engineering steps

To reverse engineer each DT in a clear and reproducible manner, we apply the same following steps to get our results:

(1) Clone the repository of the DT,
(2) Install every needed dependencies and pieces of software,
(3) Make sure that the DT works as intended,
(4) List the main characteristics of the DT,
(5) At the root of the project, list every files with a depth of n using the `tree` package[5] and copy the result into a text file,
(6) Go through every file and every folder and tag them with classes from the proposed metamodel.

The first two steps are direct. We create a local version of the repositories of the DTs on a Linux-operated machine and install the required programs and dependencies for the DT. Linux is necessary because of driver sharing issues between openHAB and Windows or macOS. During this step we encountered versioning issues mostly due to new versions breaking compatibility. Then, once installed,

[4] as methods ensure state consistency in the OOP paradigm.
[5] https://man.archlinux.org/man/tree.1.en

we familiarize ourselves with the DTs to make sure they work as intended. In the fourth step of our process, we collect and compare the main characteristics of our uses-cases. As from the presentation, our examples are pretty diverse as they represent various reference systems and use very different tools and software. The fifth and sixth steps of this process are manual. Using the terminal, we execute the following `tree` command at the root of the repository of each DT:

```
tree -L n . > <dt_name>-tree.txt
```

Here, the `-L` option is used with *n* to tell the package that we only want to show files and folders to a depth up to *n* (*n* being different from DT to DT) and we redirect the output to a `.txt` file. We then manually tag the files in the trees by their functionality in the DT using the classes of the metamodel. For example, if the file is a script, it is tagged with `Configuration` and if it is data, it is tagged with `Knowledge`.

4.2 Results and alignment of the use cases

Following the reverse engineering steps, we showcase the file trees of the HUT and Mass-Spring-Damper DTs in Figure 4 and Figure 3. The trees have different sizes with 7,582 directories and 50,738 files for the Robotic Arm DT; 691 directories and 849 files for the HUT DT; and 7 directories and 19 files for the MSD DT[6]. To reach a manageable number of files to work with, we set the depth level to (*n*=5) for the Robotic Arm and (*n*=6) for HUT, providing us with respectively 71 directories and 50 files, and 47 directories and 117 files. These are the lowest depth levels which give us at least the same number of files to work with compared to the MSD UC. As in Figure 4, we skip some parts of the trees for better clarity and readability (represented in the figure by `[...]`), therefore showing less files and directories in the figures.

In the Mass-Spring-Damper tree, four of the seven classes of the metamodel are used. This DT is organized in the following way: at the root are 4 files mainly focused towards `Configuration`, and 3 directories including `input` which mainly contains `Knowledge` of the DT, `bin` which contains a `jar` file (downloaded additionally) and `strategies` which contains deployment strategies for Ansible, Docker and bash scripts. Even though most folders contain files of one class, we can always identify at least one ambivalent file. Files like `docker.Makefile`, `cosim.json.j2` or `docker-compose.yml` are tagged with two classes as they contain some content belonging to one class and some other content belonging to the other class. As this DT is only a simulation, no `Connection` file is to be found.

In the HUT DT, four of the seven classes of the metamodel are used. As the tools used are different, this DT is organized differently from the Mass-Spring-Damper. Indeed, at the root of the DT are two files: a README and a `compose.yml` mostly configuring the DT. We also find one directory containing a configuration backup for the HUT Docker container. As in the MSD DT, we come across many ambivalent files. Any `Connection` file is present in the file tree, we suspect the files regarding this concern to be located in the HUT Docker container.

In the robotic arm DT, six of the seven classes are used. This DT is organized differently from the other two: a `Sys_de_Ref` directory

[6] All the generated trees can be found at https://github.com/labsticc-p4s/redt2024

```
.
├── bin ..................................... Tool[7]
├── docker.Makefile ..... Tool Configuration/Configuration
├── output
│   ├── outputs.csv ........................ Knowledge
│   └── msdi.log ........................... Knowledge
├── input
│   ├── cosim.json.j2 ....... Tool Configuration/Knowledge
│   ├── cosim.json.tpl ...................... Knowledge
│   ├── MassSpringDamper1.fmu ............... Knowledge
│   ├── MassSpringDamper2.fmu ............... Knowledge
│   └── time.json ........... Tool Configuration/Knowledge
├── LICENSE.md ............................ Configuration
├── Makefile ...................... Configuration/Service
├── README.org ............................ Configuration
└── strategies
    ├── ansible
    │   ├── ansible.cfg ..................... Configuration
    │   ├── hosts.ini ....................... Configuration
    │   ├── playbook.yml ............ Service/Configuration
    │   ├── requirements.yml ............... Configuration
    │   └── Vagrantfile ..................... Configuration
    ├── docker
    │   ├── docker-compose.yml ....... Service/Configuration
    │   ├── Dockerfile ...................... Configuration
    │   └── run.sh ............................ Service
    └── shell
        └── start.sh ........................... Service
```

Figure 3: Mass-Spring-Damper file tree

contains files about the robotic arm and a second one is focused on the representation of the Arm in Unity. `Knowledge` is placed in the reference folder, alongside a shell script to launch the ROS program. The `Configuration_DT` directory acts as a 'dump' for anything related to Unity, meaning that it contains multiple classes of files and folders. This DT is unique as it contains a `Tool` within it. Additionally, the DT can be launched in two ways: directly 'as is' on the machine or through a Docker container.

Additionally, an alignment table between the UCs and the components of the DT metamodel is proposed in Table 1. In this table, columns represent the five classes that compose the DT (those included in the dotted rectangle) and rows represent the UCs and the alignment between the files of the UCs and the components of the metamodel. This table reflects the provided trees for the three UCs. For the three DTs, we mention the name of an external component responsible for the `Service` concern. The same goes for the `Connection` part on the robotic arm and the HUT DT. Additionally, as a textbook case, the Mass-Spring-Damper DT, is not connected to any reference system hence the empty Connection cell for that DT.

For the Robotic Arm, the *ROS* external `Service` that acts as the interface between the DT and the Tool is added. For HUT, additional `Service` is offered by the *openHAB API* and openHAB provides

```
.
├── compose.yml ..................... Service/Configuration
└── oh_backups/hut/openhab
    ├── addons
    ├── backup.properties ................. Configuration
    └── conf........................... Tool Configuration
    │   ├── [...]
    │   ├── scripts [...]........Service/Tool Configuration
    │   ├── services
    │   │   ├── addons.cfg
    │   │   └── *.cfg
    │   ├── things [...]..... Knowledge/Tool Configuration
    │   └── transform
    │       └── *.map ............................... Service
    └── userdata ............................. Knowledge
        ├── [...]
        ├── backup
        │   ├── userdata-2022-12-09T16-50-23.tar
        │   └── [...]
        ├── config..................... Tool Configuration ?
        ├── jsondb
        │   └── *.json
        ├── persistence
        │   ├── [...]
        │   └── rrd4j ........................... Knowledge
        └── zwave ........................... Configuration
            └── *.xml
└── README.md............................ Configuration
```

Figure 4: HUT: an openHAB-based DT (condensed)

an integration of the Z-Wave protocol (`Connection`). An *external dashboard* provides `Services` to the MSD DT.

5 Analysis

After the reverse engineering of the three aforementioned use cases, we have observed that classifying the files composing the DT is not straightforward. Indeed, each use-case reveals a different internal organization of the components of the DT, often imposed by the set of tools and frameworks used during development. Moreover, the different categories of components (*i.e.*, classes in the metamodel of Section 3) are often entangled (*i.e.*, tool configuration scripts are often mixed with DT knowledge such as models). This heterogeneity in the organization of deployed DTs evidences the need for a reverse-engineering process helping to comprehend and manipulate them. We have also observed (as we claimed in Section 1) that once deployed, the boundaries of the DT do not exist explicitly, and thus the DT cannot be seen and used as a cohesive entity. In this same sense, we have found that, while a variety of scripts exist, some used to deploy the DT, some used to install, configure, and/or run it, the life cycle of the DTs is just implicit.

It is worth noticing that we have used as the target of our reverse-engineering process a very simple metamodel. We believe our metamodel could be improved. We could add new concepts refining Knowledge in Data, Model, and Metadata for instance, and Service could be specialized in Reference System services and Twin services (for management). This will make the extracted representation of our reverse engineering process richer. We see this as future work. However, even in this preliminary state, our work shows that existing DTs would benefit from re-engineering to improve their organization, and hence their understandability, and probably their maintainability and their composability. Furthermore, a higher-level reverse engineering such as a DT architecture analysis could eliminate the threat of being influenced by software engineering approaches and used technologies.

6 Related works

DT is a generic powerful concept that has the potential of becoming a systematic paradigm, when data and models are used to manage a reference system all along its life cycle. DTs are for instance regarded as a relevant design choice to implement monitoring and evolution mechanisms for CPS in DevOps infrastructures [6, 14]. DTs for DevOps is also a perspective of our work that intends, as a first step, to facilitate DevOps for DTs. [6] also underlines the still existing need for reference models. Yet, extensive work on DT conceptualization is conducted. However, the proposed DT models have different purposes and, in absence of standards, have different use and definition of concepts. To tackle this issue, [11] proposes a high-level conceptual framework, mainly focused on data and models, and shows its capacity for representing conceptual models of DT architectures all along the life cycle of the reference system. Our metamodel has the same goal but a different purpose: to propose a standard conceptual framework for DTs as software entities to facilitate their operations (deployment, execution, evolution...). Many works study how to leverage and adapt software engineering approaches to DTs, as MDE and DevOps [13, 16, 18]. They propose solutions to forward engineer DTs and are often tied to specific tools and ecosystems. On the contrary, our work is generic and tailored to the re-engineering of deployed digital twins. It therefore intends to support the management of the technical sustainability of DTs, that, as an extension of the sustainability of the reference systems, is now regarded as one of the main requirement for CPS design and operation [9].

7 Conclusion and future work

We have presented a reverse engineering work on three DTs. These DTs were developed with different goals, and using different technologies and approaches. We have discussed the need for such a reverse engineering process and advocated for the reification of DTs. In our view, a first class entity DT is a software component that can be managed as a whole, as a single entity. In this sense, we propose to separate the DT in two parts: a knowledge management part, the DT itself, and an exploitation part, implemented by various kinds of tools. This separation of responsibilities could foster tool re-usability and enable knowledge management through DTs. It should become effective through re-engineering existing *ad hoc* implementations of DTs.

Digital Twin	Service	Knowledge	Connection	Configuration	Tool Configuration
Robotic Arm	launch_pick_and_ place.sh *.sh *ROS* Management (*folder*)	*.bag.active	catkin_ws (*folder*) Resources (*folder*) Executable (*folder*) *.debug Unity_Robot_Stream (*folder*)	launch_pick_[...].sh README.md Dockerfile Resources (*folder*) DeploymentConfigu-ration	Resources (*folder*) Unity_Robot_Stream (*folder*)
HUT	compose.yml scripts (*folder*) *.map *openHAB API*	things (*folder*) userdata (*folder*) rrd4j (*folder*)	openHAB integration of Z-Wave	compose.yml backup.properties zwave (*folder*) README.md	conf (*folder*) scripts (*folder*) things (*folder*) config (*folder*)
MSD	Makefile playbook.yml docker-compose.yml run.sh start.sh *external dashboard*	outputs.csv msdi.log cosim.json.j2 cosim.json.tpl MassSpringDamper1.fmu MassSpringDamper2.fmu time.json		docker.Makefile LICENSE.md Makefile README.org ansible.cfg docker-compose.yml playbook.yml	docker.Makefile cosim.json.j2 time.json

Table 1: Alignment between the use cases and the proposed metamodel. Purple elements appear in more than one column.

As further work, we intend to define a domain specific language (DSL) for Digital Twin description and management. This language will allow us to describe the content of a DT (the `Knowledge` including its `Configurations`) and its behavior all along its life cycle (`Service`). We believe this language will be of primary interest for DevOps people who develop and maintain DTs.

8 Acknowledgments

Gwendal Beaumont's Ph.D. is funded by the Futur & Ruptures program by Fondation Mines-Télécom and TSN CARNOT institute. Authors thank, Gaëlic Béchu, from IMT Atlantique, and Quentin Perez, from IRISA, who respectively worked on first versions of the Robotic Arm and HUT Digital Twins and, Prasad Talasila, from Aarhus University, who provided the MSD use case.

References

[1] Gaëlic Bechu, Antoine Beugnard, Caroline Gl Cao, Quentin Perez, Christelle Urtado, and Sylvain Vauttier. 2022. A Software Engineering Approach to Digital Twin Architecture : The HUman at home projecT and Twin Cooperation of roBot and human project case studies. ETFA 2022.

[2] Gaëlic Bechu, Antoine Beugnard, Caroline Gl Cao, Quentin Perez, Christelle Urtado, and Sylvain Vauttier. 2022. A software engineering point of view on digital twin architecture. In *ETFA 2022*. Stuttgart, Germany.

[3] Antoine Beugnard. 2023. A software engineering perspective on digital twin: many candidates, none elected. In *2023 IEEE Smart World Congress*. IEEE, 1–8.

[4] Pascal Bibow, Manuela Dalibor, Christian Hopmann, Ben Mainz, Bernhard Rumpe, David Schmalzing, Mauritius Schmitz, and Andreas Wortmann. 2020. Model-driven development of a digital twin for injection molding. In *International Conference on Advanced Information Systems Engineering*. Springer, 85–100.

[5] Francis Bordeleau, Benoit Combemale, Romina Eramo, Mark Van Den Brand, and Manuel Wimmer. 2020. Towards model-driven digital twin engineering: Current opportunities and future challenges. In *ICSMM 2020, Bergen, Norway, June 25–26*. Springer, 43–54.

[6] Benoit Combemale, Jean-Marc Jézéquel, Quentin Perez, Didier Vojtisek, Nico Jansen, Judith Michael, Florian Rademacher, Bernhard Rumpe, Andreas Wortmann, and Jingxi Zhang. 2023. Model-Based DevOps: Foundations and Challenges. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 429–433.

[7] Manuela Dalibor, Nico Jansen, Bernhard Rumpe, David Schmalzing, Louis Wachtmeister, Manuel Wimmer, and Andreas Wortmann. 2022. A cross-domain systematic mapping study on software engineering for digital twins. *Journal of Systems and Software* 193 (2022), 111361.

[8] Manuela Dalibor, Judith Michael, Bernhard Rumpe, Simon Varga, and Andreas Wortmann. 2020. Towards a model-driven architecture for interactive digital twin cockpits. In *ER 2020*. Springer, 377–387.

[9] Istvan David and Dominik Bork. 2023. Towards a taxonomy of digital twin evolution for technical sustainability. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 934–938.

[10] Jürgen Dobaj, Andreas Riel, Georg Macher, and Markus Egretzberger. 2023. Towards DevOps for Cyber-Physical Systems (CPSs): Resilient Self-Adaptive Software for Sustainable Human-Centric Smart CPS Facilitated by Digital Twins. *Machines* 11, 10 (2023), 973.

[11] Romina Eramo, Francis Bordeleau, Benoit Combemale, Mark van Den Brand, Manuel Wimmer, and Andreas Wortmann. 2021. Conceptualizing digital twins. *IEEE Software* 39, 2 (2021), 39–46.

[12] Santiago Gil, Peter H Mikkelsen, Cláudio Gomes, and Peter G Larsen. 2024. Survey on open-source digital twin frameworks–A case study approach. *Software: Practice and Experience* (2024).

[13] Amon Göppert, Lea Grahn, Jonas Rachner, Dennis Grunert, Simon Hort, and Robert H Schmitt. 2023. Pipeline for ontology-based modeling and automated deployment of digital twins for planning and control of manufacturing systems. *Journal of Intelligent Manufacturing* (2023), 1–20.

[14] Jerome Hugues, Anton Hristosov, John J Hudak, and Joe Yankel. 2020. Twinops-devops meets model-based engineering and digital twins for the engineering of cps. In *23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion (MODELS-C)*. 1–5.

[15] Dominik Hüsener, Michael Schluse, Dorit Kaufmann, and Jürgen Roßmann. 2021. The Digital Twin as a Mediator for the Digitalization and Conservation of Expert Knowledge. In *Annals of Scientific Society for Assembly, Handling and Industrial Robotics*. 241–251.

[16] Jörg Christian Kirchhof, Judith Michael, Bernhard Rumpe, Simon Varga, and Andreas Wortmann. 2020. Model-driven digital twin construction: synthesizing the integration of cyber-physical systems with their information systems. In *MODELS 2023*. 90–101.

[17] Daniel Lehner, Sabine Sint, Michael Vierhauser, Wolfgang Narzt, and Manuel Wimmer. 2021. AML4DT: A model-driven framework for developing and maintaining digital twins with automationml. In *ETFA 2021*. IEEE, 1–8.

[18] Paula Muñoz, Javier Troya, and Antonio Vallecillo. 2021. Using UML and OCL models to realize high-level digital twins. In *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 212–220.

[19] Jérôme Pfeiffer, Daniel Lehner, Andreas Wortmann, and Manuel Wimmer. 2022. Modeling capabilities of digital twin platforms-old wine in new bottles? *J. Object Technol.* 21, 3 (2022), 3–1.

[20] Concetta Semeraro, Mario Lezoche, Hervé Panetto, and Michele Dassisti. 2021. Digital twin paradigm: A systematic literature review. *Computers in Industry* 130 (2021), 103469.

[21] Daniel Stahl. 2023. https://medium.com/@danstahl1138/metadata-driven-development-a-new-paradigm-for-data-engineering-4689801558d5.

[22] Prasad Talasila, Cláudio Gomes, Peter Høgh Mikkelsen, Santiago Gil Arboleda, Eduard Kamburjan, and Peter Gorm Larsen. 2023. Digital twin as a service (DTaaS): a platform for digital twin developers and users. In *2023 IEEE Smart World Congress (SWC)*. IEEE, 1–8.